

Kernels, Cokernels, Images, and Coimages in Abelian Categories

N-20251109

§1.1 The setting: maps with zero maps

The uploaded note studies a morphism

$$f : A \rightarrow B$$

from the categorical viewpoint. The words domain and codomain mean exactly what they mean for an ordinary function:

$$A = \text{domain}, \quad B = \text{codomain}.$$

For linear maps one may think of A as the input space and B as the declared output space. The actual values reached by f may form a smaller object inside B ; that smaller object is the image.

The categorical definitions below need a zero object and zero morphisms. A zero object is simultaneously initial and terminal. Once a category has a zero object 0 , every pair of objects X, Y has a distinguished zero morphism

$$0_{X,Y} : X \rightarrow 0 \rightarrow Y.$$

This lets equations such as $f \circ k = 0$ make sense without mentioning elements.

The most familiar examples are:

$$\mathbf{Vect}_k, \quad \mathbf{Ab}, \quad R\text{-Mod}.$$

These are abelian categories. In them, kernels, cokernels, images, and coimages behave like the objects from linear algebra and module theory.

§1.2 Kernel: the universal object killed by a map

For a morphism $f : A \rightarrow B$, a kernel of f is a morphism

$$k : K \rightarrow A$$

such that

$$f \circ k = 0,$$

and universal with this property: whenever $k' : K' \rightarrow A$ also satisfies $f \circ k' = 0$, there is a unique morphism $u : K' \rightarrow K$ with

$$k' = k \circ u.$$

In diagram form:

$$\begin{array}{ccccc} K' & & & & \\ \downarrow u & \searrow k' & & & \\ K & \xrightarrow{k} & A & \xrightarrow{f} & B. \end{array}$$

The condition is that both routes from K' to B are zero after applying f .

In vector spaces this recovers

$$\text{Ker}(f) = \{a \in A : f(a) = 0\}.$$

The categorical definition says something stronger than just listing elements: the kernel is the best, largest, and universal way to factor all inputs that become zero.

In an abelian category, the kernel morphism $k : K \rightarrow A$ is a monomorphism. It is therefore legitimate to think of K as a subobject of the domain A , though the invariant datum is the arrow k , not merely the object K .

§1.3 Cokernel: the universal quotient killing a map

The cokernel is the dual construction. A cokernel of $f : A \rightarrow B$ is a morphism

$$c : B \rightarrow C$$

such that

$$c \circ f = 0,$$

and universal with this property: whenever $c' : B \rightarrow C'$ satisfies $c' \circ f = 0$, there is a unique morphism $u : C \rightarrow C'$ with

$$c' = u \circ c.$$

The diagram is:

$$\begin{array}{ccccc} A & \xrightarrow{f} & B & \xrightarrow{c} & C \\ & & & \searrow c' & \downarrow u \\ & & & & C'. \end{array}$$

In vector spaces or modules,

$$\text{Coker}(f) = B / \text{Im}(f).$$

Thus the cokernel measures what remains in the codomain after the actual output of f has been collapsed to zero.

In an abelian category, the cokernel morphism $c : B \rightarrow C$ is an epimorphism. It is therefore the categorical version of a quotient object of the codomain.

§1.4 Image: the part of the codomain actually reached

In linear algebra the image is

$$\text{Im}(f) = \{f(a) : a \in A\} \subseteq B.$$

In an abelian category, one defines the image without element language by first taking the cokernel

$$c : B \rightarrow \text{Coker}(f)$$

and then taking the kernel of that cokernel:

$$\text{Im}(f) = \text{Ker}(B \xrightarrow{c} \text{Coker}(f)).$$

So the image is represented by a monomorphism

$$i : \text{Im}(f) \rightarrow B.$$

The diagram is:

$$\text{Im}(f) \xhookrightarrow{i} B \xrightarrow{c} \text{Coker}(f).$$

The equation $c \circ i = 0$ says that everything in the image is killed by the quotient map c . The universal property says that it is the largest subobject of B killed by c .

This gives a useful slogan:

$$\text{Im}(f) = \text{the part of the codomain not visible to the cokernel.}$$

§1.5 Coimage: the effective part of the domain

The coimage is dual to the image. Begin with the kernel

$$k : \text{Ker}(f) \rightarrow A.$$

Then define the coimage as the cokernel of the kernel:

$$\text{Coim}(f) = \text{Coker}(\text{Ker}(f) \xrightarrow{k} A).$$

So it is represented by an epimorphism

$$q : A \rightarrow \text{Coim}(f).$$

The diagram is:

$$\text{Ker}(f) \xhookrightarrow{k} A \xrightarrow{q} \text{Coim}(f).$$

In vector spaces this is the quotient

$$\text{Coim}(f) = A / \text{Ker}(f).$$

It is the domain after removing precisely the part that f sends to zero. In that sense, it is the effective input object of f .

§1.6 The standard factorization

In an abelian category, every morphism $f : A \rightarrow B$ factors through its coimage and image:

$$A \xrightarrow{q} \text{Coim}(f) \xrightarrow{\bar{f}} \text{Im}(f) \xrightarrow{i} B.$$

Here:

- $q : A \rightarrow \text{Coim}(f)$ is the cokernel of $\text{Ker}(f) \rightarrow A$, hence an epimorphism;
- $i : \text{Im}(f) \rightarrow B$ is the kernel of $B \rightarrow \text{Coker}(f)$, hence a monomorphism;
- the middle map $\bar{f} : \text{Coim}(f) \rightarrow \text{Im}(f)$ is an isomorphism in an abelian category;
- the original morphism is recovered by

$$f = i \circ \bar{f} \circ q.$$

A compact diagram is:

$$\text{Ker}(f) \hookrightarrow A \xrightarrow{q} \text{Coim}(f) \xrightarrow[\cong]{\bar{f}} \text{Im}(f) \hookrightarrow B \xrightarrow{c} \text{Coker}(f).$$

f

This is the categorical form of the first isomorphism theorem:

$$A / \text{Ker}(f) \cong \text{Im}(f).$$

The uploaded note's phrase that coimage is effective input and image is actual output is exactly this factorization: q discards useless input, i embeds the actual output into the declared codomain, and $\bar{f}$ identifies these two descriptions.

§1.7 A vector-space sanity check

Let $T : V \rightarrow W$ be a linear map. Then

$$\text{Ker}(T) = \{v \in V : T(v) = 0\}, \quad \text{Coker}(T) = W / \text{Im}(T).$$

The coimage is

$$\text{Coim}(T) = V / \text{Ker}(T),$$

and the induced map

$$\bar{T} : V / \text{Ker}(T) \rightarrow \text{Im}(T), \quad [v] \mapsto T(v),$$

is an isomorphism. It is well-defined because if $v - v' \in \text{Ker}(T)$, then $T(v) = T(v')$. It is injective because $T(v) = 0$ means $[v] = 0$, and it is surjective by definition of image.

For a matrix $A : k^n \rightarrow k^m$, this says that the rank is the dimension of both sides of the middle isomorphism:

$$\dim(k^n / \text{Ker } A) = \dim \text{Im } A = \text{rank } A.$$

This is rank-nullity in categorical clothing.

§1.8 A module example: multiplication by an integer

Consider the group homomorphism

$$f : \mathbb{Z} \rightarrow \mathbb{Z}, \quad f(x) = nx.$$

If $n \neq 0$, then

$$\text{Ker}(f) = 0, \quad \text{Im}(f) = n\mathbb{Z}, \quad \text{Coker}(f) = \mathbb{Z}/n\mathbb{Z}.$$

The coimage is

$$\text{Coim}(f) = \mathbb{Z}/0 \cong \mathbb{Z}.$$

The middle isomorphism is

$$\mathbb{Z} \xrightarrow{\sim} n\mathbb{Z}, \quad x \mapsto nx.$$

This example is useful because the image is literally a subgroup of the codomain, while the coimage is a quotient of the domain. They look different as subobject and quotient, but in an abelian category they are canonically isomorphic through f .

§1.9 Exactness language

Kernel and image are also the grammar of exact sequences. A sequence

$$X \xrightarrow{g} Y \xrightarrow{h} Z$$

is exact at Y when

$$\text{Im}(g) = \text{Ker}(h)$$

as subobjects of Y . The equality means that the elements, vectors, or generalized inputs that h kills are exactly those that came from g .

In categorical language, exactness replaces element chasing by universal properties. For example, saying that

$$\text{Ker}(f) \rightarrow A \rightarrow B$$

is exact at A is precisely saying that the first arrow is the kernel of f . Saying that

$$A \rightarrow B \rightarrow \text{Coker}(f)$$

is exact at B is precisely saying that the second arrow is the cokernel of f , after passing through the image.

§1.10 Warnings: what depends on being abelian

The statements above should not be read as valid in every category. Several layers are being used.

- A category must have a zero object and zero morphisms before kernels and cokernels can even be formulated in this form.
- A category may have kernels and cokernels but still fail to make every monomorphism a kernel or every epimorphism a cokernel.
- The canonical map $\text{Coim}(f) \rightarrow \text{Im}(f)$ need not be an isomorphism outside abelian categories.

Thus the beautiful factorization

$$\text{Coim}(f) \cong \text{Im}(f)$$

is not a formal tautology. It is one of the defining strengths of abelian categories.

For intuition, **Set** is not the right environment for this theory: it has no zero object in the needed sense. Pointed sets have a zero object and kernels in a pointed sense, but they are not abelian. Vector spaces, abelian groups, and modules are the safe models.

§1.11 Synthesis: from element chasing to universal properties

The definitions fit into the following table:

object	categorical construction	linear algebra model	where it lives
$\text{Ker}(f)$	universal map into A killed by f	$\{a : f(a) = 0\}$	subobject of A
$\text{Coker}(f)$	universal quotient of B killing f	$B / \text{Im}(f)$	quotient of B
$\text{Im}(f)$	$\text{Ker}(B \rightarrow \text{Coker}(f))$	$\{f(a) : a \in A\}$	subobject of B
$\text{Coim}(f)$	$\text{Coker}(\text{Ker}(f) \rightarrow A)$	$A / \text{Ker}(f)$	quotient of A

The standard factorization

$$A \twoheadrightarrow \operatorname{Coim}(f) \xrightarrow{\sim} \operatorname{Im}(f) \hookrightarrow B$$

turns a morphism into three conceptual pieces: quotient out the part of the input killed by f , identify the remaining effective input with the subobject actually reached in B , and then include that subobject in the codomain.

For vector spaces this says exactly the familiar rank–nullity picture: $A/\ker f$ is isomorphic to $\operatorname{im} f$. For modules and abelian groups the same statement is the first isomorphism theorem. The categorical formulation is useful because it records which part of the argument is universal and which part depends on being in an abelian category: the comparison map $\operatorname{Coim}(f) \rightarrow \operatorname{Im}(f)$ is an isomorphism in the safe models listed above, but it should not be assumed in an arbitrary category with kernels and cokernels.